

Temporal abstraction in reinforcement learning

With thanks to Rich Sutton, Satinder Singh, Pierre-Luc Bacon, Jean Harb

What is temporal abstraction?

- Consider an activity such as cooking dinner



- High-level steps: choose a recipe, make a grocery list, get groceries, cook,...
- Medium-level steps: get a pot, put ingredients in the pot, stir until smooth, check the recipe ...
- Low-level steps: wrist and arm movement while driving the car, stirring, ...
- All have to be seamlessly integrated!
- Cf. macro actions in classical AI, controllers in robotics

Formalization of temporal abstraction

- Hierarchical abstract machines (Parr, 1998)
- MAXQ (Dietterich, 1998)
- Dynamic motion primitives (Schaal et al. 2004)
- Skills (Konidaris et al, 2009)
- *Options* (Sutton, Precup & Singh, 1999; Precup, 2000)

Options framework

- Suppose we have an MDP $\langle \mathcal{S}, \mathcal{A}, r, P, \gamma \rangle$
- An *option* ω consists of 3 components
 - An *initiation set* of states $I_\omega \subseteq \mathcal{S}$ (aka precondition)
 - A *policy* $\pi_\omega : \mathcal{S} \times \mathcal{A} \rightarrow [0, 1]$
 $\pi_\omega(a|s)$ is the probability of taking a in s when following option ω
 - A *termination condition* $\beta_\omega : \mathcal{S} \rightarrow [0, 1]$:
 $\beta_\omega(s)$ is the probability of terminating the option ω upon entering s
- Eg., robot navigation: if there is no obstacle in front (I_ω), go forward (π_ω) until you get too close to another object (β_ω)

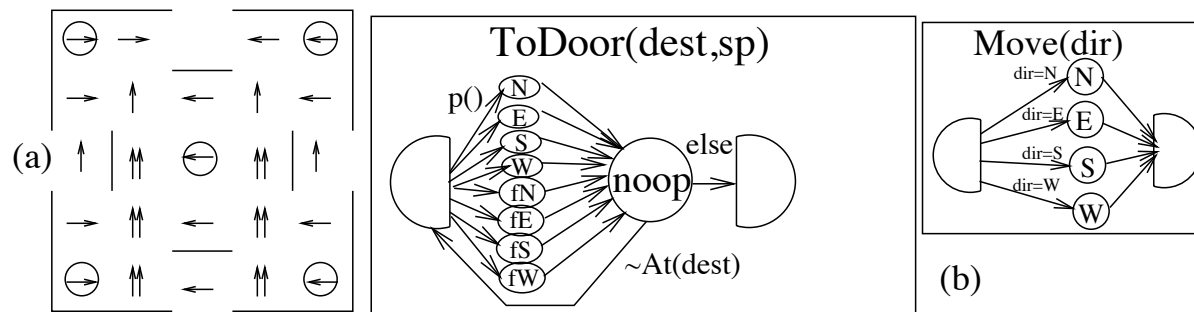
Cf. Sutton, Precup & Singh, 1999; Precup, 2000

Options as behavioral programs

- *Call-and-return execution*
 - Option is a subroutine which gets called by a policy over options π_{Ω}
 - When called, ω is pushed onto the execution stack
 - During the option execution, the program looks at certain *variables (aka state)* and executes an *instruction (aka action)* until a termination condition is reached
 - The option can keep track of additional *local variables*, eg counting number of steps, saturation in certain features (e.g. Comanici, 2010)
 - *Options can invoke other options*
- *Interruption*
 - At each step, one can check if a better alternative has become available
 - If so, the option currently executing is *interrupted* (special form of concurrency)
- *The option identity is also a form of memory: what is the agent currently trying to achieve?* Cf. Shaul et al, 2014, Kulkarni et al, 2016

Alternative formalisms

- MAXQ (Dietterich, 2000): specify a graph of sub-tasks
- Hierarchies of Abstract Machines (Parr & Russell, 1997): abstractions given by automata
- Andre (2003): programming language (ALISP) which allows specifying HAM programs, learning parts of them



Option models

- *Option model* has two parts:
 1. *Expected reward* $r_\omega(s)$: the expected return during ω 's execution from s
 - Needed because it is used to update the agent's internal representations
 2. *Transition model* $P_\omega(s'|s)$: a sub-probability distribution over next states (reflecting the discount factor γ and the option duration) given that ω executes from s
 - P specifies *where* the agent will end up after the option/program execution and *when* termination will happen
- Models are *predictions* about the future, conditioned on the option being executed

Option models provide semantics

- Programming languages: preconditions (initiation set) and postconditions
- *Models of options represent (probabilistic) post-conditions*
- *Models that are compositional*, can be used to reason about the policy over options
- *Sequencing*

$$\mathbf{r}_{\omega_1\omega_2} = \mathbf{r}_{\omega_1} + P_{\omega_1}\mathbf{r}_{\omega_2}$$

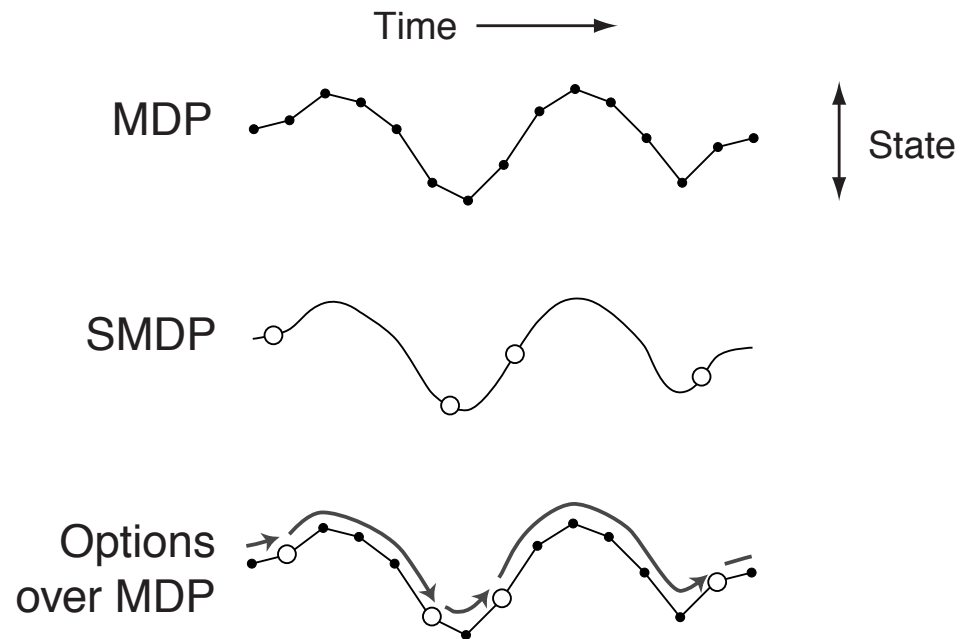
$$P_{\omega_1\omega_2} = P_{\omega_1}P_{\omega_2}$$

Cf. Sutton et al, 1999, Sorg & Singh, 2010

- *Stochastic choice*: can take expectations of reward and transition models
- These are sufficient conditions to allow Bellman equations to hold
- Silver & Ciosek (2012): re-write model in one matrix, compose models to construct programs

Eg. good generalization in Towers of Hanoi

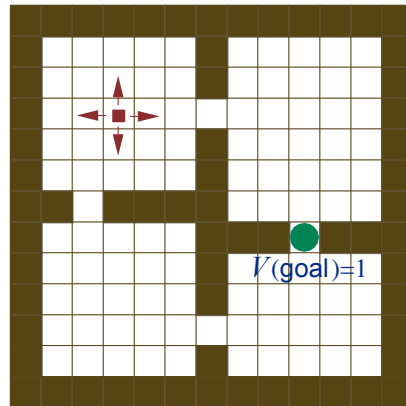
MDP + Options = Semi-Markov Decision Process



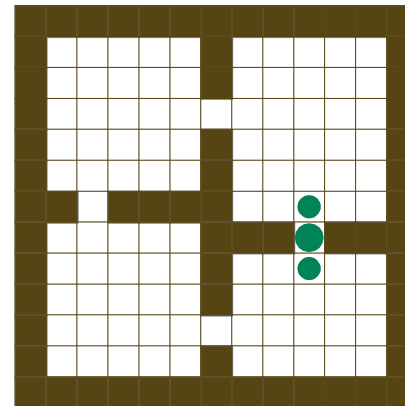
- Introducing options in an MDP induces a related semi-MDP
- Hence *all planning and learning algorithms* from classical MDPs transfer directly to options (Cf. Sutton, Precup & Singh, 1999; Precup, 2000)
- But planning and learning with options can be much faster!

Illustration: Navigation

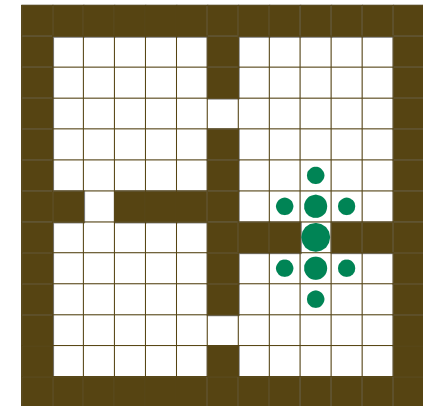
with cell-to-cell
primitive actions



Iteration #0

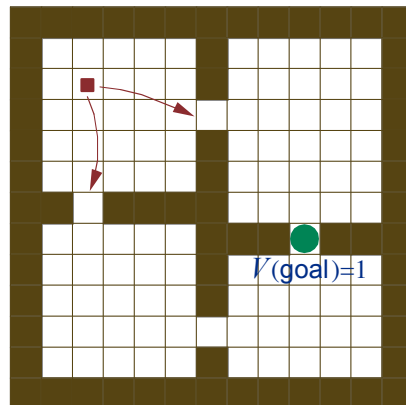


Iteration #1

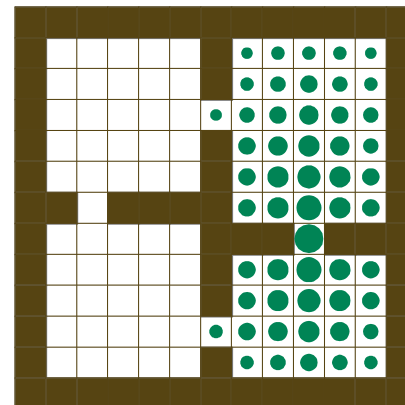


Iteration #2

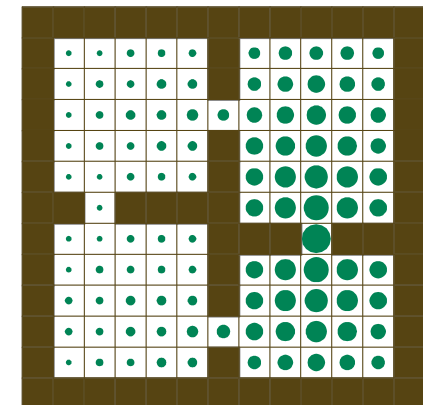
with room-to-room
options



Iteration #0



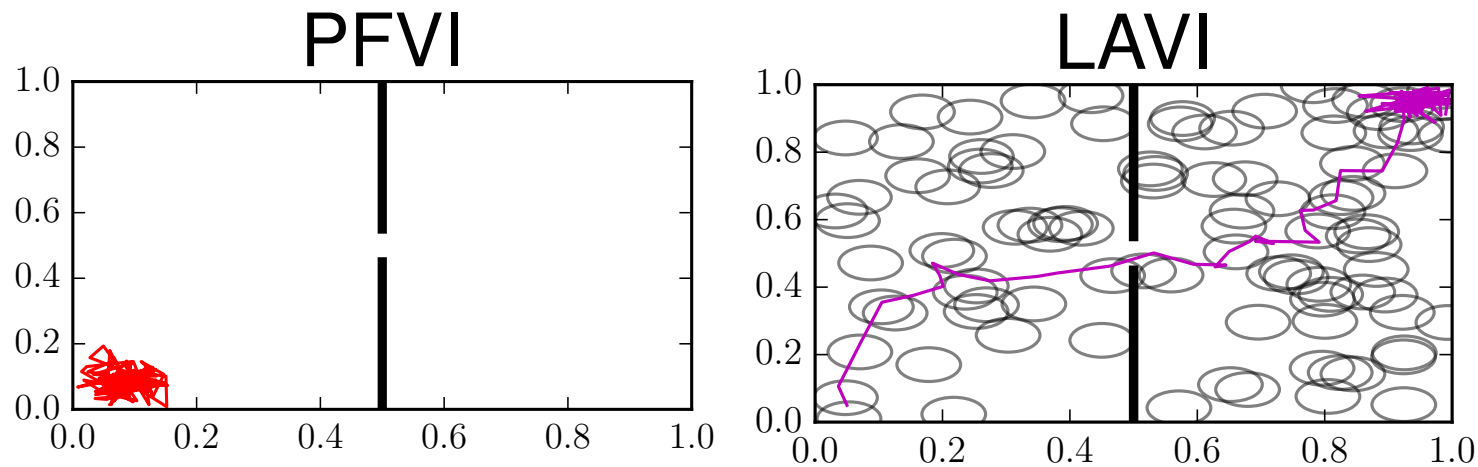
Iteration #1



Iteration #2

Illustration: Random landmarks

- *Generate a lot of options, then worry about which are useful!*
- Large set of *landmarks*, i.e. states in the environment, chosen at random (Mann, Mannor & Precup, 2015)
- Rough planner which can get to a landmark from its vicinity, by solving a *deterministic relaxation* of the MDP



Landmark-based approximate value iteration gets a good solution much faster!

The anatomy of the reward option model

- Primitive action model: $r_a(s) = \mathbb{E}[r_t | s_t = s, a_t = a]$
- Option model:

$$r_\omega(s) = \mathbb{E}[r_t + \gamma r_{t+1} + \dots | s_t = s, \omega_t = \omega]$$

- This expectation indicates a Markov-style property, as it depends only on the identity of the state and the option, not on the time step
- Notice the *model is basically a value function* so we can write Bellman equations for the model:

$$r_\omega(s) = \sum_a \pi_\omega(s, a) [r(s, a) + \sum_{s'} \gamma (1 - \beta_\omega) r_\omega(s')]$$

- This means that we can use RL methods to learn the models of options!
- Very similar equations hold for the transition model

Intra-option algorithms

- Learning about one option at a time is very inefficient
- In fact, we may not want to execute options at all!
- Instead, *learn about all options consistent with the behaviour*
- In some sense, a form of *attention*
- E.g. action-value function, tabular case

On single-step transition $\langle s, a, r, s' \rangle$, for all ω that could have been executing in s and taken a :

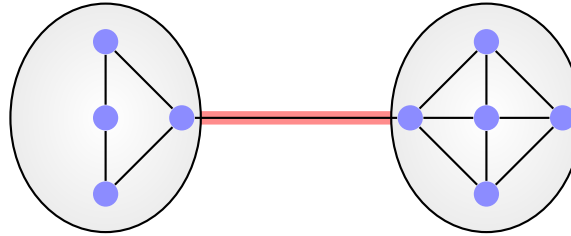
$$\begin{aligned} Q_{\Omega}(s, \omega) &= Q_{\Omega}(s, \omega) + \alpha[r_a(s) + \gamma(1 - \beta_{\omega}(s'))Q_{\Omega}(s', \omega) + \\ &+ \beta_{\omega}(s')\gamma \sum_{s'} \max_{\omega'} Q_{\Omega}(s', \omega') - Q_{\Omega}(s, \omega)] \end{aligned}$$

- In general function approximation, importance sampling will need to be used (several papers on this)

Frontier: Option Discovery

- Options can be given by a system designer
- If subgoals / secondary reward structure is given, the option policy can be obtained, by solving a smaller planning or learning problem (cf. [Precup, 2000](#))
- *What is a good set of subgoals / options?*
- This is a *representation discovery* problem
- Studied a lot over the last 15 years
- Bottleneck states and change point detection currently the most successful methods

Bottleneck states

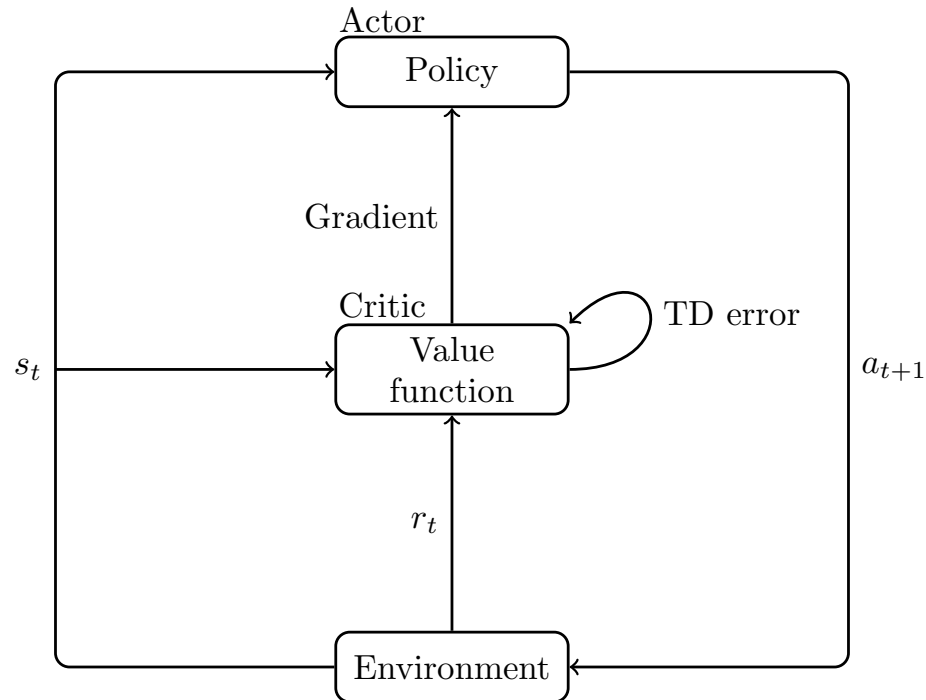


- Perhaps the most explored idea in options construction
- A bottleneck is a special state, which is visited more often than others, allows “circulating on the graph”
- Lots of different approaches!
 - Frequency of states (McGovern et al, 2001, [Stolle & Precup, 2002](#))
 - Graph partitioning / state graph analysis (Simsek et al, 2004, Menache et al, 2004, [Bacon & Precup, 2013](#))
 - Information-theoretic ideas (Peters et al., 2010)
- People seem quite good at generating these (cf. Botvinick, 2001, Solway et al, 2014)
- *Main drawback: expensive both in terms of sample size and computation*

Option-critic

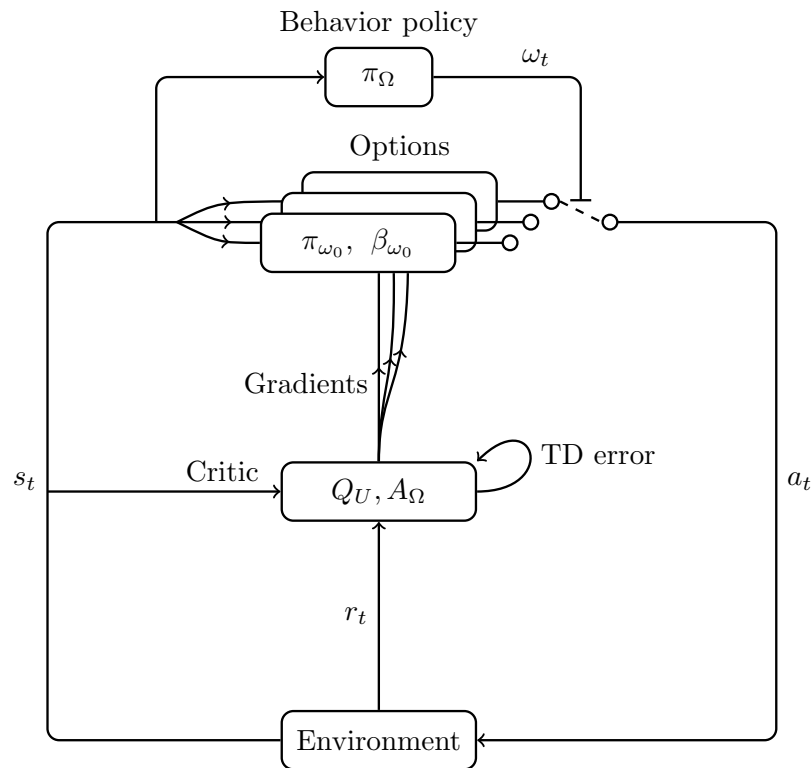
- Explicitly state an *optimization objective* and then solve it to find a set of options
- Handle both *discrete and continuous* set of state and actions
- Learning options should be *continual* (avoid combinatorially-flavored computations)
- Options should provide *improvement within one task* (or at least not cause slow-down...)

Actor-critic architecture



- Clear optimization objective: average or discounted return
- Continual learning
- Handles both discrete and continuous states and actions

Option-critic architecture



- Parameterize internal policies and termination conditions
- Policy over options is computed by a separate process (planning, RL, ...)

Formulation

- The option-value function of a policy over options π_Ω is given by

$$Q_{\pi_\Omega}(s, \omega) = \sum_a \pi_\omega(a|s) Q_U(s, \omega, a)$$

where

$$Q_U(s, \omega, a) = r_a(s) + \gamma \sum_{s'} P_a(s'|s) U(\omega, s')$$

- The last quantity is the utility from s' onwards, *given that we arrive in s' using ω*

$$U(\omega, s') = (1 - \beta_\omega(s')) Q_{\pi_\Omega}(s', \omega) + \beta_\omega(s') V_{\pi_\Omega}(s')$$

- We parameterize the internal policies by θ , as $\pi_{\omega, \theta}$, and the termination conditions by ν , as $\beta_{\omega, \nu}$
- *Note that θ and ν can be shared over the options!*

Main result: Gradient updates

- Suppose we want to optimize the expected return: $\mathbb{E} \{Q_{\pi_{\Omega}}(s, \omega)\}$
- The *gradient wrt the internal policy parameters* θ is given by:

$$\mathbb{E} \left\{ \frac{\partial \log \pi_{\omega, \theta}(a|s)}{\partial \theta} Q_U(s, \omega, a) \right\}$$

This has the usual interpretation: *take better primitives more often* inside the option

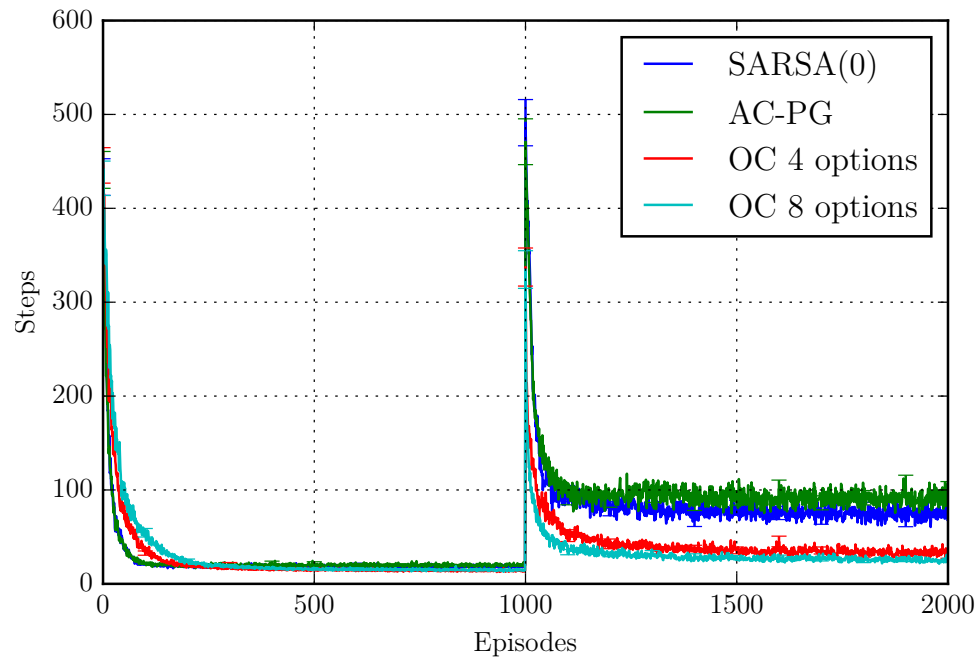
- The *gradient wrt the termination parameters* ν is given by:

$$\mathbb{E} \left\{ -\frac{\partial \beta_{\omega, \nu}(s')}{\partial \nu} A_{\pi_{\Omega}}(s', \omega) \right\}$$

where $A_{\pi_{\Omega}} = Q_{\pi_{\Omega}} - V_{\pi_{\Omega}}$ is the advantage function

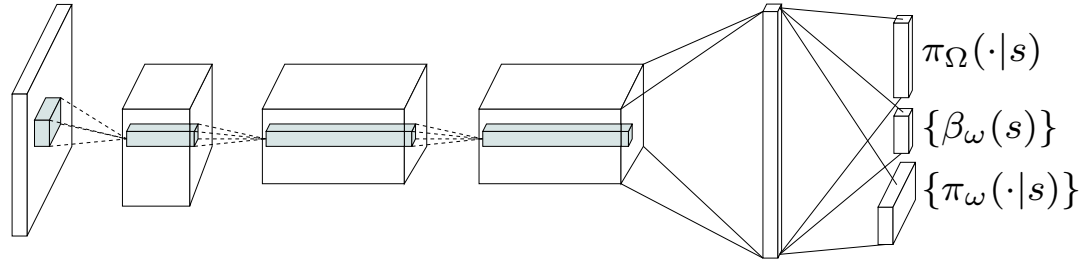
This means that we want to *lengthen options that have a large advantage*

Results: Options transfer

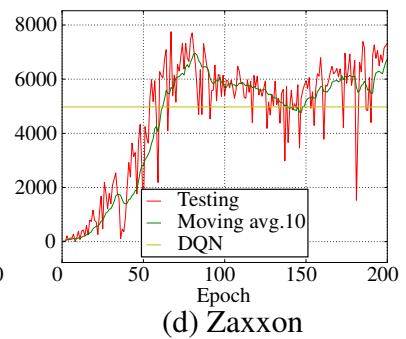
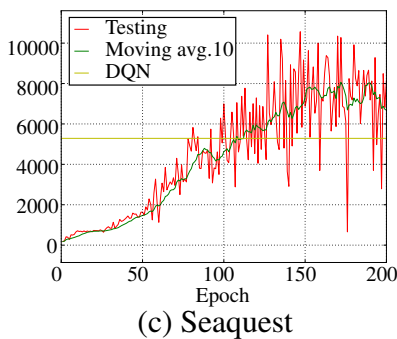
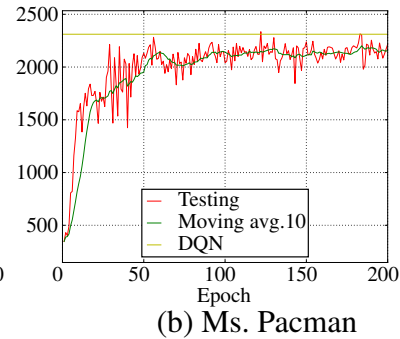
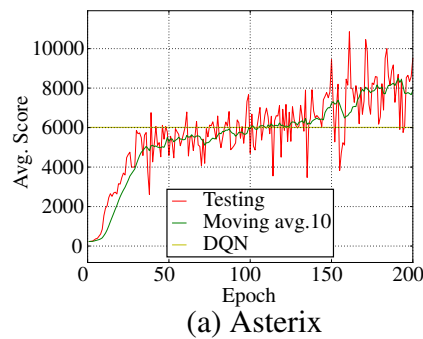


- 4-rooms domain, tabular representations, value functions learned by Sarsa
- Learning in the first task no slower than using primitives
- Learning once the goal is moved faster with the options

Results: Nonlinear function approximation



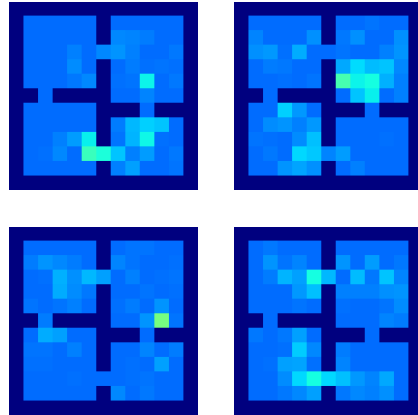
- Atari simulator, DQN to learn value function over options, actor as above



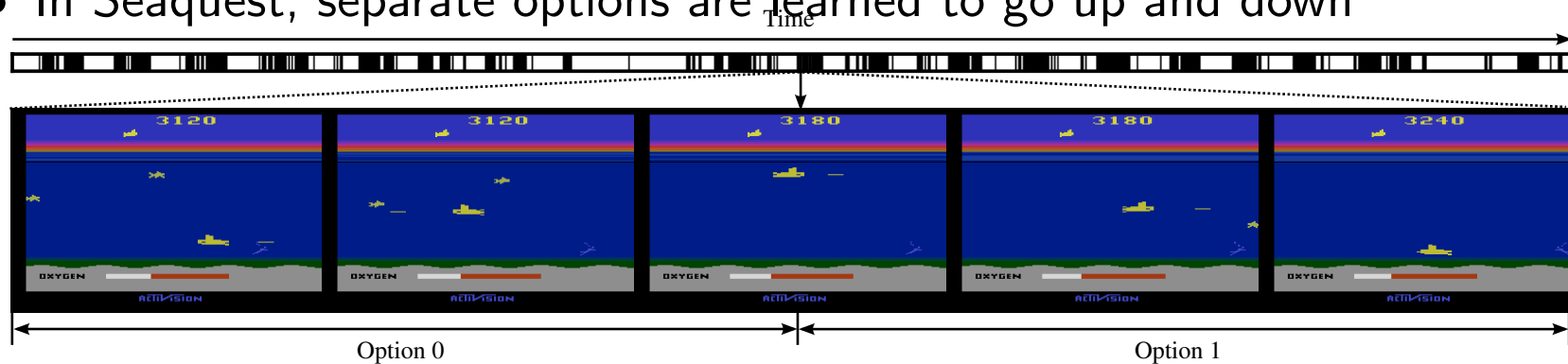
- Performance matching or better than DQN

Results: Learned options are intuitive

- In rooms environment, terminations are more likely near hallways (although there are no pseudo-rewards provided)



- In Seaquest, separate options are learned to go up and down



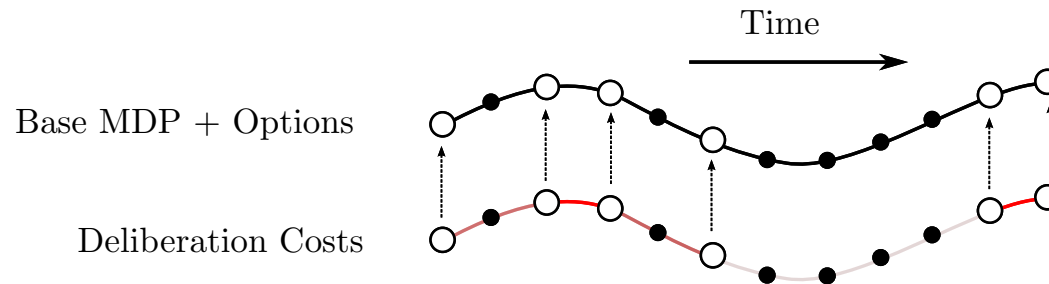
What are beneficial options

- Successful simultaneous learning of terminations and option policies
- But, as expected, *options shrink over time* unless a margin is required for the advantage
Cf. time-regularized options, Mann et al, (2014)
- Intuitively, using longer options increase the speed of learning and planning (but may lead to a worse result in call-and-return execution)
- What is the right tool to formalize this intuition?

Regularization

- Entropy-based regularization is useful to encourage diversity in the options
- *Length collapse*: options “dissolve” into primitive action over time
- Assumption: *executing a policy is cheap, deciding what to do is expensive*
 - Many choices may need to be evaluated (branching factor over actions)
 - In planning, many next states may need to be considered (branching factor over states)
 - Evaluating the function approximator might be expensive (e.g. if it is a deep net)
- Deliberation is also expensive in animals:
 - Energy consumption (to engage higher-level brain function)
 - Missed opportunity cost: thinking too long means action is delayed

Deliberation/Switching Cost



- Let $c(s, \omega) < 0$ be the immediate cost of switching to ω in s
- *New value function expresses total deliberation cost:*

$$Q_c(s, \omega) = c(s, \omega) + \sum_{s'} P(s'|s, \omega) \sum_{\omega'} \mu(\omega'|s') Q_c(s', \omega')$$

- *New objective: maximize reward with reasonable effort*

$$\max_{\Omega} \mathbb{E} [Q_{\Omega}(s, \omega) + \tau Q_c(s, \omega)]$$

$\tau \geq 0$ controls the trade-off between value and computation effort

- Can be shown equivalent to requiring that *advantage exceeds a threshold* before switching

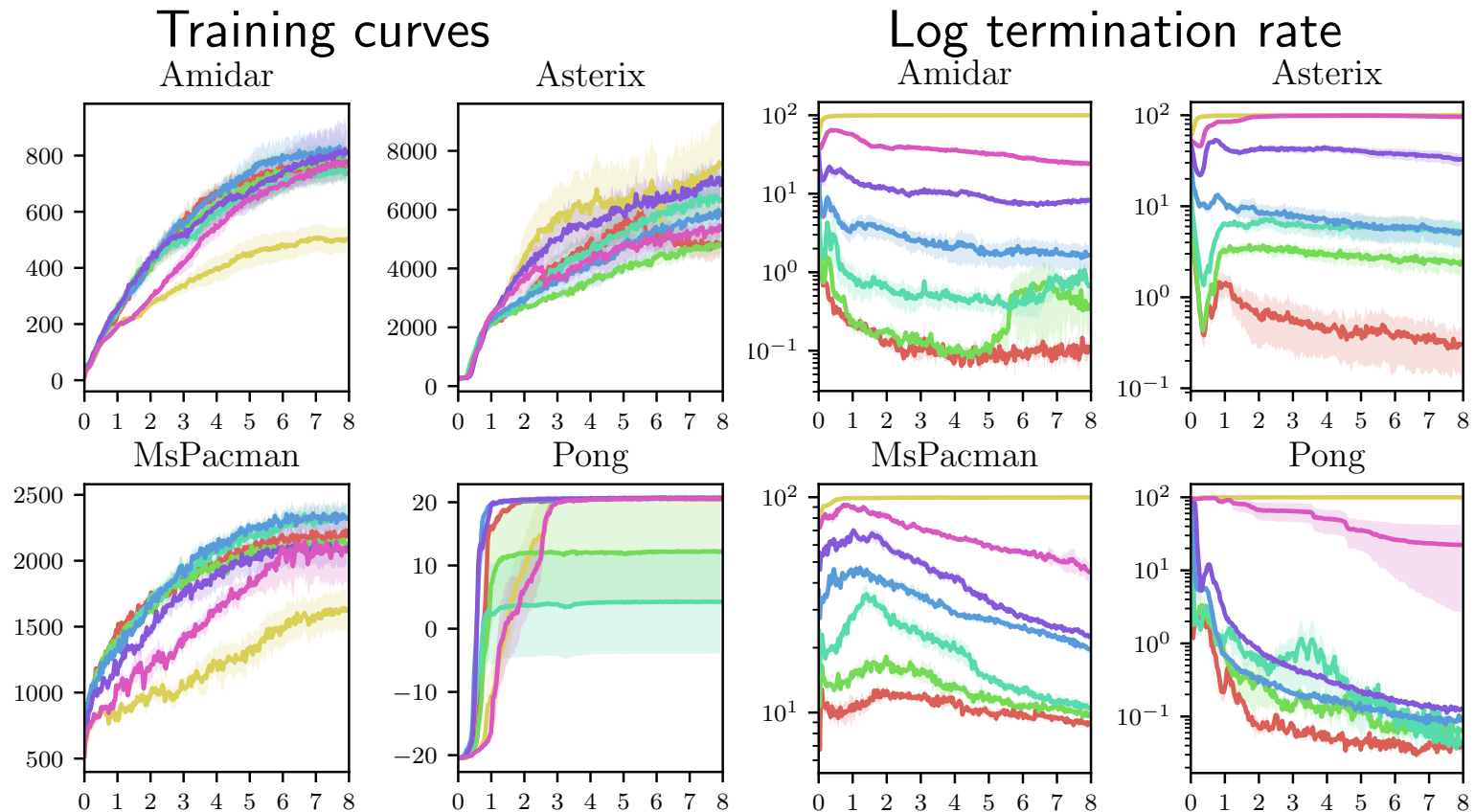
Interesting properties

- Immediate cost of deliberation is computed based on properties of the environment
- We do not need pseudo-rewards for sub-goals!
- Value function over options is still obtained accurately
- If $\xi = 0$ we simply optimize returns as before

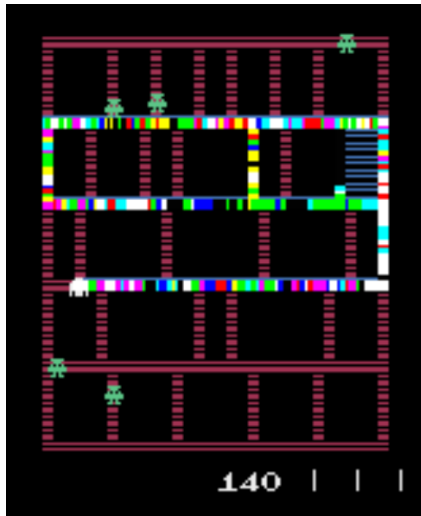
Example: Immediate cost = number of actions

- With primitive actions: average cost of $|\mathcal{A}|$ per time step
- With options only: average cost of $|\Omega|$ incurred *only when re-deciding what to do*
- If we re-decide on average every k th step, and if $|\Omega| < k|\mathcal{A}|$, deliberation with options is cheaper
- Even if we use both options and primitive actions, average deliberation is cheaper if $|\Omega| < (k - 1)|\mathcal{A}|$

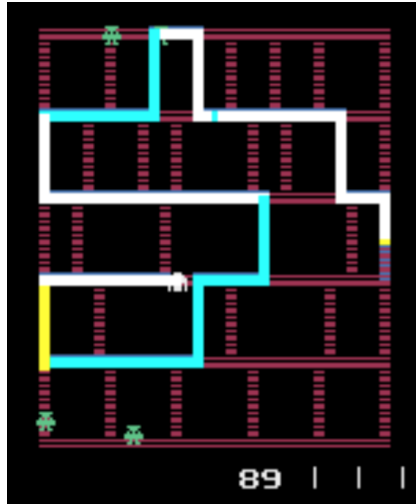
Effect of Deliberation Cost Regularization



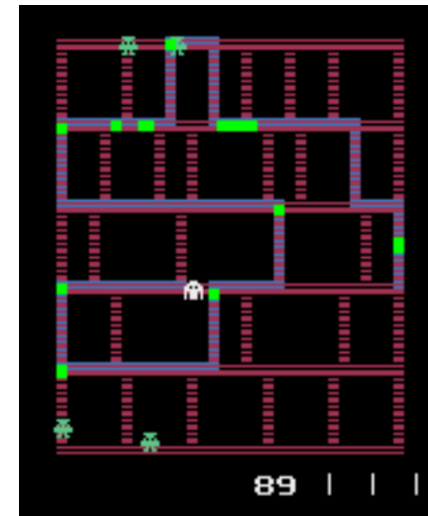
Example: Amidar



(a) Without a deliberation cost, options terminate instantly and are used in any scenario without specialization.



(b) Options are used for extended periods and in specific scenarios through a trajectory, when using a deliberation cost.



(c) Termination is sparse when using the deliberation cost. The agent terminates options at intersections requiring high level decisions.

- Deliberation costs prevent options from becoming too short
- Terminations are intuitive

Relationship to other ideas

- Human problem solving: Solway et al (2014) proposed a Bayesian model selection framework to explain subgoal learning by humans trying to navigate an unknown map
- Humans found subgoals “around” bottleneck states
- Inspection of their criterion (Bayesian fit to the data) shows strong similarity with using a cost equal to the branching factor between options, plus the branching factor within the active option
- *Deliberation costs can also explain the value of options for exploration*
 - Travelling quickly around the environment means values will become accurate more quickly
 - The best action becomes clear earlier, which would make it easier to choose

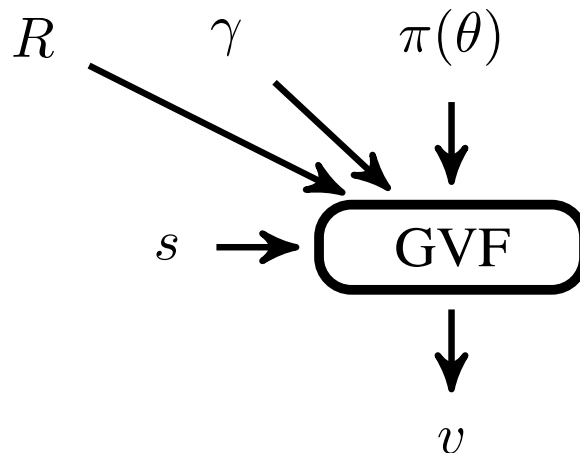
Knowledge representation: Generalized Value Functions (GVFs)

- Given a cumulant function c , state-dependent continuation function γ and policy π , the Generalized Value Function $v_{\pi,\gamma,c}$ is defined as:

$$v_{\pi,\gamma,c}(s) = \mathbf{E} \left[\sum_{k=t}^{\infty} C_{k+1} \prod_{i=t+1}^k \gamma(S_i) \mid S_t = s, A_{t:\infty} \sim \pi \right]$$

- Cumulant c** is a function that can depend on the S_t , or (S_t, A_t) or (S_t, A_t, S_{t+1}) and can output a vector (even a matrix)
Eg $C_{k+1} = c(S_k, A_k, S_{k+1})$
- Continuation function γ** maps states to $[0,1]$
- Cf. Horde architecture (Sutton et al, 2011); Adam White's thesis; inspiration from Pandemonium architecture

GVPs will be the building blocks of knowledge



- Note that one can take the output of a GVP and make it an input to another GVP
- Or, the output of a GVP could become part of the “state” for another GVP

Usual value functions are GVFs

- Cumulant is reward, γ is constant

$$\begin{aligned} v_{\pi}(s) &= \mathbf{E}\left[\sum_{k=t}^{\infty} R_{k+1} \gamma^{k-t} \mid S_t = s, A_{t:\infty} \sim \pi\right] \\ &= \mathbf{E}\left[\sum_{t=0}^{\infty} R_{t+1} \prod_{i=t+1}^k \gamma \mid S_t = s, A_{t:\infty} \sim \pi\right] \end{aligned}$$

- A special case is when a policy is optimal wrt $c, \gamma, v_{c,\gamma}^*$ - Universal Value Function approximation (UVFA) (Schaul et al, 2015)

Successor states and successor features are GVFs

- *Successor features* (Barreto et al, 2017, 2018) are a natural extension of successor states (Dayan, 1992)
- Successor states give the expected occupancy of future states
- If states are defined by a feature vector $\phi(s)$, successor features give the expected, discounted sum of future feature vectors from a state.
- In GVF terms, the cumulant is $c = \phi$, and there is a fixed policy and discount
- Interesting property highlighted in Barreto et al:

$$v_{\mathbf{w}^T c, \pi, \gamma}(s) = \mathbf{w}^T v_{c, \pi, \gamma}(s)$$

which leads to very efficient use of existing GVFs to do one-shot computation of new GVFs

The anatomy of the reward option model

- Option reward model is defined as:

$$r_{\omega}(s) = \mathbb{E}[R_{t+1} + \gamma R_{t+2} + \dots | S_t = s, \omega_t = \omega]$$

We can write Bellman equations for the model:

$$r_{\omega}(s) = \sum_a \pi_{\omega}(s, a) [r(s, a) + \sum_{s'} P_{\omega}(s'|s) \gamma (1 - \beta_{\omega}(s')) r_{\omega}(s')]$$

- This means the **option reward model is a GVF**, $v_{\omega,r}$: cumulant is the environment reward r , policy is π_{ω} , continuation function is $\gamma(1 - \beta_{\omega})$
- Note that **we could use $\gamma = 1$ for the overall task** and these GVFs would still be well defined
- Option transition model can similarly be shown to be a GVF

Option-value function

- The option-value function of a policy over options π_Ω is defined as:

$$q_{\pi_\Omega}(s, \omega) = \mathbf{E}_{\pi_\Omega} [R_{t+1} + \gamma\beta_\omega(S_{t+1})q_{\pi_\Omega}(S_{t+1}, \omega_{t+1})] \\ + \gamma((1 - \beta_\omega(S_{t+1}))q_{\pi_\Omega}(S_{t+1}, \omega)|S_t = s]$$

- *Option-value function is a GVF* but the cumulant includes the value of the next options
- Intuitively, models are more self-contained than option-value functions

To do list

- Maybe: improve off-policy learning
- How do we generate behavior from GVFs ?
 - Option call-and-return execution gives a hint, but is too limiting
 - Ideally, we will use GVFs to plan (done for option models so far)
 - Exploration: how to generate data for many GVFs?
- Discovery: how do we get initial conditions, policies and continuation/termination functions?